

Algorithms

Implementation: `core/expert_tree.py` , `domains/geochem/experts/*.py` ,
`domains/geochem/sources/*.py` , `domains/geochem/prospectivity_model.py` ,
`domains/geochem/narrative.py` .

Notation. x — query location; S — active data sources; E — experts; P — known-deposit (positive) samples; B — background samples; $\phi_s(x)$ — feature map of source s ; μ, σ — background mean and standard deviation; w — feature weight (domain prior); $d \in \{-1, +1\}$ — learned direction; ω_s — source weight; W_c — expert weight; $\sigma(\cdot)$ — logistic function.

Algorithm 1 Feature extraction — $\phi_s(x)$

Input : location x , data source s

Output: feature map f

```
if  $s$  is a geochemical assay source then      # 5 media: stream sediment,
  for  $r \in \{5, 10, 50\}$  km do                # rockchip, drillhole,
     $N_r \leftarrow \text{KDTree}(s).\text{query\_ball}(x, r)$     # shallow drill, soil
    for each element  $e$  do
       $v \leftarrow \text{log1p}(\text{concentrations of } e \text{ in } N_r)$ , discard non-finite and negative
      if  $|v| < \text{MIN\_SAMPLES}$  then  $f[\cdot] \leftarrow \text{NaN}$ 
      else
         $f[e\_r\text{km\_median}] \leftarrow \text{median}(v)$ 
         $f[e\_r\text{km\_p90}] \leftarrow \text{percentile}(v, 90)$ 
         $f[e\_r\text{km\_cv}] \leftarrow \text{std}(v) / \text{mean}(v)$ 
         $f[e\_r\text{km\_frac\_above}] \leftarrow \text{mean}(v > \theta_e)$  #  $\theta_e$ : 75th pct of  $e$  over WA
     $f[n\_samples\_r\text{km}] \leftarrow |N_r|$ 
  for each element  $e$  do
     $f[e\_contrast\_5\_50] \leftarrow f[e\_5\text{km\_median}] - f[e\_50\text{km\_median}]$ 
     $f[e\_contrast\_10\_50] \leftarrow f[e\_10\text{km\_median}] - f[e\_50\text{km\_median}]$ 
    # a difference in log space is the log of a concentration ratio:
    # local enrichment relative to the regional background
  for each configured pair  $(a,b)$  do
     $f[\text{ratio\_}a\_b] \leftarrow f[a\_10\text{km\_median}] - f[b\_10\text{km\_median}]$ 
   $f[\text{spatial\_coherence\_}10\text{km}] \leftarrow \text{mean pairwise distance of the 5 highest values}$ 
   $f[\text{PC}\{i\}\_5,10\text{km\_median}] \leftarrow \text{median of neighbourhood PCA scores}$ 

else if  $s$  is a geophysical raster source then
  for name  $\in \{\text{mag, grav, K, Th, U, LuHf, SmNd}\}$  do
     $f[\text{name}] \leftarrow \text{raster sampled at } x$ 
     $f[\text{name\_grad}] \leftarrow \text{sqrt}(dx^2 + dy^2)$       # horizontal gradient magnitude
   $f[K\_Th], f[Th\_U], f[U\_K] \leftarrow \text{corresponding ratios}$ 

else if  $s$  is a structural / geological vector source then
   $f[\text{dist\_fault\_km}], f[\text{fault\_density\_}5\text{km}], f[\text{fault\_density\_}10\text{km}]$ 
   $f[\text{dist\_worm\_mag\_km}], f[\text{dist\_worm\_grav\_km}]$ 
   $f[\text{is\_rt\_}]$  (lithology one-hot),  $f[\text{age\_ma}], f[\text{is\_yilgarn}], f[\text{is\_pilbara}], f[\text{is\_covered}]$ 
```

```
return f
```

Algorithm 2 Leaf expert — fit and score

```
procedure EXPERT.FIT(P, B)
  for each source  $s \in \text{required\_sources}$  do
     $F_P \leftarrow \{\phi_s(x) : x \in P\}$ ;  $F_B \leftarrow \{\phi_s(x) : x \in B\}$ 
    if  $F_B = \emptyset$  then continue
    for each feature  $k$  do
       $\mu_s[k], \sigma_s[k] \leftarrow \text{mean, std of } F_B[k]$     #  $\sigma < 1e-9 \Rightarrow \text{set to } 1$ 
       $w_s[k] \leftarrow \text{domain prior}$                 # equal for enrichment experts;
                                                    # pathfinder prior dict otherwise
       $d_s[k] \leftarrow +1$  if  $\text{median}(F_P[k]) \geq \text{median}(F_B[k])$  else  $-1$     # learned from data
     $a \leftarrow \text{AUC}(\text{score of source } s \text{ alone on } P \cup B)$ 
    if  $a < 0.5$  then  $a \leftarrow 1 - a$ ;  $d_s \leftarrow -d_s$     # source is anti-correlated
     $\omega_s \leftarrow \max(0, (a - 0.5) \times 2)$                 # source weight
  end for

function EXPERT.SCORE(x)
  for each source  $s$  with  $\omega_s > 0$  do
     $f \leftarrow \phi_s(x)$ ; if  $f = \perp$  then continue
     $z_s[k] \leftarrow (f[k] - \mu_s[k]) / \sigma_s[k] \cdot d_s[k]$     # direction folded into  $z$ 
     $\text{raw}_s \leftarrow \sum_k w_s[k] \cdot z_s[k] / \sum_k w_s[k]$     # finite  $k$  only: a NaN feature
                                                    # enters neither numerator nor
                                                    # denominator
    if no finite term then continue
     $p_s \leftarrow \sigma(\text{raw}_s)$ 
  if no source usable then return  $\perp$                 # ABSTAIN: the expert leaves
                                                    # the aggregation entirely

   $\text{score} \leftarrow \sum_s \omega_s \cdot p_s / \sum_s \omega_s$ 
  # chain-normalised per-feature weights, so that  $\sum w \cdot z$  reconstructs  $\text{raw}_s$ 
   $W_{\text{feat}}[s:k] \leftarrow (\omega_s / \sum \omega) \cdot (w_s[k] / \sum_{\{k' \text{ present in } s\}} w_s[k'])$ 
  return NodeScore(score, feature_z = z, feature_w =  $W_{\text{feat}}$ , weights =  $\omega$ )
```

Algorithm 3 Expert tree — fit and score

```
procedure COMPOSITENODE.FIT(P, B)
  for each child expert  $c \in E$  do
     $c.FIT(P, B)$ 
   $v \leftarrow [c.SCORE(x).score : x \in P \cup B, \text{score} \neq \perp]$ 
  if  $|v| < 10$  or  $v$  has a single class then  $W_c \leftarrow 0$ ; continue
   $a \leftarrow \text{AUC}(v, \text{labels})$ 
  if  $a < 0.5$  then  $a \leftarrow 1 - a$ ;  $c.invert \leftarrow \text{true}$     # each node owns exactly one
                                                    # flip of its own output;
                                                    # no ancestor re-applies it
   $W_c \leftarrow \max(0, (a - 0.5) \times 2)$                 # global weight
```

```

function COMPOSITENODE.SCORE(x)
  C ← { c ∈ E : c.SCORE(x) ≠ ⊥ }           # abstaining experts vanish here
  if C = ∅ then return ⊥
  for c ∈ C do e_c ← W_c · c.local_reliability(x) # default reliability 1.0
  if  $\sum e_c < 1e-9$  then
    score ← mean{ c.score : c ∈ C }; confidence ← 0
  else
    score ←  $\sum_c e_c \cdot c.score / \sum_c e_c$ 
    confidence ←  $\sum_{\{c \in C\}} e_c / \sum_{\{c \in E\}} W_c$  # fraction of fitted weight used
  if self.invert then score ← 1 – score
  return NodeScore(score, confidence, children = C, weights = e)

```

Algorithm 4 End-to-end scoring and narrative

```

procedure FIT(catalog, config, exclude_ids)
  S ← sources passing the coverage probe           # 40 random points, ≥3 samples
                                                    # within 10 km, ≥10% hit rate
  P ← known deposits(config.sites_csv) \ exclude_ids
  B ← uniform random points in the WA bounding box with ≥3 samples within 10 km,
    |B| = 300, fixed seed
  E ← 3 geochemical experts (across all assay sources)
    u 4 geophysical experts u 3 structural experts
  tree ← COMPOSITENODE(E); tree.FIT(P, B)

function EXPLAIN(x)
  ns ← tree.SCORE(x)
  tier ← Strong if ns.score ≥ 0.80, Moderate if ≥ 0.65, Weak if ≥ 0.52, else Background
  for c ∈ ns.children do
    contrib_c ←  $(e_c / \sum e) \cdot |ns.children[c].score - 0.5|$ 
  for each feature key "s:k" do
    attribution["s:k"] ←  $(e_c / \sum e) \cdot feature\_w["s:k"] \cdot feature\_z["s:k"]$ 
  return PointNarrative(score, tier, confidence,
    ranked contrib, ranked attribution)
  # a structured record, not a string: the agent layer may restate its numbers
  # but cannot introduce new ones

```

Identity (measured: exact to 0.0 on 9/9 commodities).

$$ns.score = \sum_c (e_c / \sum e) \cdot \sigma(\sum_k feature_w[c][k] \cdot feature_z[c][k])$$

Algorithm 5 Evaluation protocol

```

for each commodity do
  all_pos ← records whose SITE_CODE field matches "Mine|Deposit"
  test_pos ← first 30 of a fixed permutation (seed 42); train_pos ← remainder
  forbid ← coordinates of test_pos, in km

  # training set, shared by every model
  train_neg ← random WA points with assay coverage, ≥5 km from any test positive

```

```

(n = 300, seed 142)

# four test-negative strategies
random    ← random WA points, covered, ≥5 km from test positives (n=200, seed 43)
far_random ← as above and ≥50 km from any known deposit          (n=200, seed 44)
true_nonmine ← no_<el>_sites.csv: real deposits of other commodities
              where the target commodity is absent                (n≤200, seed 45)
spatial    ← train on south (Y < -25), test on north (Y ≥ -25);
              negatives are non-deposit sites in the north        (n=200, seed 47)

expert_tree ← FIT(train_pos, train_neg)                            # Algorithm 4
X ← feature table over all active sources, columns named "source:feature"
X ← restrict to the 195 columns the 10 experts actually read (expert_matched);
    impute NaN with the training median
for m ∈ {DecisionTree(depth 4), RandomForest(500), XGBoost(400, depth 4),
        L1-Logistic} do m.fit(X_train, y_train)

report AUC and P@{10,20,50} for every scenario

```

Algorithm 6 Explanation quality — identical protocol for all models

```

# attribution: each model supplies its own; none is imposed on another
A[expert_tree] ← (e_c / Σ e) · | feature_w · feature_z |
A[decision_tree] ← weighted impurity decrease of the nodes on the decision path
A[random_forest] ← TreeSHAP (shap.TreeExplainer)
A[xgboost]      ← booster.predict(pred_contribs = True)          # exact TreeSHAP
A[logit_l1]     ← | coef · (x - training mean) |

# faithfulness: claimed top driver vs. the feature that actually moves the score
for each feature j do                                           # all 195, no sampling
    E[i,j] ← | score(x_i) - score(x_i with feature j set to the training median) |
    # for the expert tree this passes through the real model: overwrite the cached
    # source feature, re-score, restore
Hit@1 ← mean_i [ argmax_j A[i,j] = argmax_j E[i,j] ]
ρ ← mean_i spearman( A[i,:], E[i,:] )
# group level: partition the columns by the expert that reads them, mask whole
# groups, recompute both quantities

# stability
for b = 1 ... 10 do
    resample the training set with replacement, refit, recompute A_b
    J_b ← mean_i jaccard( top3(A[i,:]), top3(A_b[i,:]) )
stability ← mean_b J_b

```

Algorithm 7 Missing modalities and self-reported reliability

```

for k = 0 ... |S| - 1 do
    for rep = 1 ... N_REP do

```

```

D ← k sources drawn at random
# expert tree: blank D at the test points ⇒ the affected experts return ⊥
#           and drop out; the remaining weights renormalise
score_expert, confidence ← tree.SCORE(test points with D blanked)
# fixed-width models cannot skip a modality; the columns must be imputed
score_ml ← m.predict(X with columns of D ← training median)
# XGBoost also run with native NaN
record AUC(k), mean confidence(k), number of active experts(k)

# selective prediction: each model ranked by its own reliability signal
signal[expert_tree] ← confidence # independent of the score
signal[others] ← | p − 0.5 | # a transform of the score;
# RF additionally: std over trees
for coverage ∈ {100%, 75%, 50%, 25%} do
  retain the highest-signal fraction of points and recompute AUC
p_err ← spearman( signal, −| y − p | )

```